

DIA 2

LangChain e Python: criando ferramentas

Do loop artesanal para componentes declarativos, validados e testáveis

1

Por que o LangChain existe

2

@tool, Pydantic e erros que ensinam

3

Structured output com retry

4

RAG local sobre a pasta docs/

As oito horas de hoje

1

09:00 – 10:30 • A dor de ontem e as ferramentas

Por que o LangChain existe, model providers, e o decorador `@tool` substituindo o schema escrito à mão.

2

10:45 – 12:15 • Laboratório: migrar o NEXUS

Converter as ferramentas de ontem, rodar o loop novo, contar as linhas que sumiram.

3

13:15 – 14:45 • Structured output, LCEL e o que há sob o RAG

Objetos em vez de texto, composição com o operador pipe, callbacks e streaming, e a matemática mínima da busca.

4

15:00 – 16:30 • Laboratório: RAG local de verdade

Indexar docs/, criar `buscar_documentos`, responder uma pergunta que exige buscar E calcular.

5

16:30 – 17:00 • Fechamento

Erros clássicos, o caderno de falhas e o gancho do dia 3.

01

A dor de ontem

Duas ferramentas funcionam. Trinta, não.

O que quebra com 30 ferramentas e 4 pessoas

1

Duplicação entre função e schema

Alguém renomeia o argumento na função e esquece o JSON. O modelo passa a mandar um nome que não existe, e o erro só aparece em execução.

2

Nenhuma validação de tipo

O modelo manda expressão igual a 42, um número, e a função quebra com `TypeError` no meio do loop.

3

Tratamento de erro sem padrão

Cada pessoa escreve do seu jeito. Uns levantam exceção, outros devolvem `None`, e o agente reage diferente a cada um.

4

Impossível testar isolado

Para testar uma ferramenta é preciso subir o agente inteiro e torcer para o modelo chamá-la.

O ecossistema, separado em partes

Pacote	O que faz	Usamos?
langchain-core	Mensagens, ferramentas, runnables, prompts	sim, o tempo todo
langchain	create_agent, middlewares, conveniência	sim
langchain-ollama / -openai	Integrações com provedores	sim
langchain-chroma / -huggingface	Vector store e embeddings	sim
langgraph	Orquestração em grafo de estado	dia 3 em diante
langsmith	Observabilidade hospedada	opcional, só mencionamos

O que o LangChain NÃO é

1 Não é framework multiagente

Coordenação de vários agentes é LangGraph, a partir do dia 3. O LangChain sozinho monta um agente.

2 Não é servidor

Não expõe API, não escala, não persiste nada por conta própria. Isso continua sendo seu problema de arquitetura.

3 Não substitui entender o harness

Ele implementa partes das cinco camadas. Decidir o que entra no contexto, quando parar e o que verificar continua com você.

O modelo, em uma linha

A mesma abstração serve para Ollama, llama.cpp e vLLM. Muda o import e a URL.

Regra que vale o curso inteiro: toda função de negócio recebe llm como parâmetro. Nunca instancie o modelo dentro dela.

É isso que permite trocar de backend, rodar teste com um modelo falso e comparar dois modelos na mesma execução.

```
from langchain_ollama import ChatOllama

llm = ChatOllama(model="qwen3:4b", temperature=0)
print(llm.invoke("Diga apenas: ok").content)

# apontando para llama.cpp ou vLLM:
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="Qwen/Qwen3-8B",
    base_url="http://localhost:8000/v1",
    api_key="nao-usado",
    temperature=0,
)
```

.env: nunca uma chave no código

O curso é local-first, mas o ChatOpenAI é o mesmo cliente dos dois mundos: muda a URL e a chave. Vale formalizar a forma correta de guardar essa chave, porque é o erro de segurança mais comum em projeto de LLM.

`load_dotenv()` lê o `.env` e popula os valores em `os.environ`. A chave nunca aparece no código.

Em produção as variáveis vêm do orquestrador (Docker, secret do Kubernetes), não de um `.env` no disco.

```
$ pip install python-dotenv
$ echo "OPENAI_API_KEY=sk-..." > .env
$ echo ".env" >> .gitignore

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI

load_dotenv()

llm = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0,
)
# a chave vem do ambiente, nunca de
# uma string literal no código
```


Trocar de provedor sem tocar no agente

Backend	Classe	O que muda
Ollama	ChatOllama	model apenas
llama.cpp	ChatOpenAI	base_url para a porta 8080 e model livre
vLLM	ChatOpenAI	base_url para a porta 8000 e o id do modelo no Hub
OpenAI	ChatOpenAI	só a chave; sem base_url
Hugging Face local	ChatHuggingFace	carrega o modelo no processo; dia 5

02

Ferramentas com @tool

Uma função. O schema sai das anotações.

O decorador @tool

```
from langchain.tools import tool

@tool
def calcular(expressao: str) -> str:
    """Avalia uma expressao aritmetica e devolve o resultado exato.

    Use SEMPRE que houver conta, mesmo simples.

    Args:
        expressao: expressao aritmetica, ex: '950 * 24'
    """
    return str(_avaliar(ast.parse(expressao, mode="eval").body))

print(calcular.name)                # calcular
print(calcular.description)         # a docstring
print(calcular.args_schema.model_json_schema()) # o JSON que ontem voce escreveu a mao
```

O schema sai das anotações de tipo. A descrição sai da docstring. A duplicação de ontem desaparece.

A mesma ferramenta, ontem e hoje

D1 Dia 1: à mão

- Função Python separada do schema JSON
- Nome repetido em três lugares
- Nenhuma validação antes de executar
- Testar exige subir o agente
- Cerca de 20 linhas por ferramenta

D2 Dia 2: com @tool

- Uma função anotada, e nada mais
- Nome definido uma vez só
- Pydantic valida antes da execução
- `calcular.invoke({...})` testa isolado
- Cerca de 8 linhas por ferramenta

O que economiza horas depois

1 Anotações são obrigatórias

Sem expressão: str o schema sai vazio e o modelo inventa os argumentos. É o bug mais comum de quem começa, e ele não dá erro: só faz o agente errar.

2 snake_case e sem acento

Alguns provedores rejeitam outros formatos, e modelos pequenos erram mais com nomes exóticos. Vale para nome de ferramenta e de argumento.

3 A docstring é prompt

Ela vai literalmente para dentro do contexto do modelo. Escreva-a para o modelo, não para o colega: diga QUANDO usar, não só o que faz.

Duas descrições da mesma ferramenta

X O modelo ignora

- "Busca documentos."
- "Função de busca semântica no índice."
- "Retorna uma lista de trechos."
- Descreve implementação
- Não diz em que situação usar

OK O modelo usa na hora certa

- "Busca trechos na base local de documentos."
- "Use ANTES de afirmar qualquer coisa sobre o conteúdo dos documentos."
- "Se a resposta depender de um documento, chame esta ferramenta primeiro."
- Descreve gatilho de uso
- Cabe em três linhas

Pydantic quando os argumentos são mais que uma string

Ganhos concretos, não decorativos:

ge=1, le=10 impede o modelo de pedir k=5000 e estourar o contexto.

Literal vira um enum no schema, e o modelo passa a escolher entre três valores em vez de inventar o quarto.

Se vier k igual a "quatro", o Pydantic devolve erro de validação ANTES de a função rodar.

```
from pydantic import BaseModel, Field
from typing import Literal

class BuscaArgs(BaseModel):
    consulta: str = Field(
        description="Termos em linguagem natural")
    k: int = Field(default=4, ge=1, le=10,
        description="Quantos trechos retornar")
    fonte: Literal["docs", "notas", "todos"] = Field(
        default="todos", description="Onde procurar")

@tool(args_schema=BuscaArgs)
def buscar_documentos(consulta: str, k: int = 4,
    fonte: str = "todos") -> str:
    """Busca trechos relevantes na base local."""
    ...
```

ToolRuntime: o que o modelo não vê

Às vezes a ferramenta precisa de algo que o modelo não deve escolher: quem chamou, uma conexão de banco, a configuração do usuário.

O parâmetro runtime NÃO aparece no schema mandado ao modelo. Ele é injetado pelo framework.

Em código antigo você verá InjectedState e InjectedStore. ToolRuntime é a interface única que substituiu as duas.

```
from langchain.tools import tool, ToolRuntime

@tool
def ultima_pergunta(runtime: ToolRuntime) -> str:
    """Recupera a ultima pergunta do usuario."""
    for m in reversed(runtime.state["messages"]):
        if m.type == "human":
            return m.content
    return "nenhuma"

# o modelo ve apenas:
#   ultima_pergunta() -> sem argumentos
# o runtime chega por injecao, nao por
# decisao do LLM.
```


Duas classes de erro, dois tratamentos

R Recuperável pelo modelo

- Arquivo não existe
- Argumento fora do intervalo
- Consulta sem resultado
- Expressão com letra no meio
- Devolva string começando com ERRO: dizendo o que fazer diferente

N Não recuperável

- Banco de dados fora do ar
- Credencial inválida
- Disco cheio
- Bug no seu código
- Levante a exceção e deixe o harness decidir se tenta de novo ou aborta

Ferramenta virou código testável

```
# testes/test_ferramentas.py
import pytest
from ferramentas import calcular, ler_arquivo

def test_calcula_certo():
    assert calcular.invoke({"expressao": "950 * 24"}) == "22800"

def test_rejeita_expressao_invalida():
    saida = calcular.invoke({"expressao": "import os"})
    assert saida.startswith("ERRO:")

def test_nao_sai_da_pasta():
    saida = ler_arquivo.invoke({"caminho": "../../etc/passwd"})
    assert "fora da pasta" in saida

def test_erro_lista_alternativas():
    saida = ler_arquivo.invoke({"caminho": "nao_existe.md"})
    assert "ERRO" in saida and ".md" in saida      # sugere os arquivos que existem
```

Nenhum modelo foi carregado para rodar estes quatro testes. Eles rodam em milissegundos, no CI, a cada commit.

Quantas ferramentas o modelo aguenta

Quantidade	O que acontece	O que fazer
Até 5	Escolha quase sempre certa	Nada; siga em frente
6 a 12	Começam confusões entre nomes parecidos	Nomes distintos e description com gatilho
13 a 25	Erro de escolha visível, sobretudo em modelo pequeno	Agrupar por especialidade, ou filtrar por etapa
Acima de 25	O bloco de schemas domina o contexto	Dividir em agentes especialistas (dia 4)

Três perguntas antes do intervalo

1

O modelo nunca chama a sua ferramenta nova

Onde você olha primeiro?

2

O modelo manda k=5000 e o contexto estoura

O que faltou no schema?

3

A ferramenta quebrou e o agente morreu

Era erro recuperável ou não?

03

0 loop, de novo

O que sumiu e o que ficou.

O mesmo loop, agora com objetos do LangChain

```
llm_com_tools = llm.bind_tools([calcular, ler_arquivo, buscar_documentos])
REGISTRO = {t.name: t for t in FERRAMENTAS}

def rodar(pergunta: str, max_passos: int = 8):
    msgs = [SystemMessage(SISTEMA), HumanMessage(pergunta)]
    for _ in range(max_passos):
        ai = llm_com_tools.invoke(msgs)
        msgs.append(ai)
        if not ai.tool_calls:
            return ai.content
        for tc in ai.tool_calls:
            saida = REGISTRO[tc["name"]].invoke(tc["args"])
            msgs.append(ToolMessage(content=str(saida), tool_call_id=tc["id"]))
    return "limite de passos atingido"
```

Sumiu: montar schema à mão, json.loads dos argumentos, validação de tipo. Ficou: exatamente o que é decisão de projeto.

O que o framework assumiu e o que continua com você

Responsabilidade	Ontem	Hoje
Montar o schema JSON	você, à mão	derivado da função
Converter argumentos	json.loads manual	já vem como dict
Validar tipos	não havia	Pydantic, antes de executar
Formato da mensagem de ferramenta	dict montado à mão	ToolMessage
Decidir quando parar	você	você
Decidir o que entra no contexto	você	você
Verificar se a resposta presta	você	você

Os quatro tipos de mensagem

Classe	Quem produz	Para que serve
SystemMessage	você	Instruções, papel, formato de saída
HumanMessage	o usuário	A pergunta ou o comando
AIMessage	o modelo	Texto de resposta ou tool_calls
ToolMessage	o seu executor	O resultado real, amarrado por tool_call_id

E o create_agent?

O LangChain traz um atalho que embrulha esse loop inteiro em uma linha.

E é justamente por isso que ele não foi ensinado primeiro.

Quem vê create_agent no primeiro contato acha que agente é uma função pronta. Quem escreveu o loop duas vezes sabe o que essa linha esconde, e sabe onde intervir.

Use para protótipo. A partir do dia 3, montamos o grafo explicitamente.

```
from langchain.agents import create_agent

agente = create_agent(
    model=llm,
    tools=[calcular, ler_arquivo],
    system_prompt=SISTEMA,
)

r = agente.invoke({
    "messages": [HumanMessage("Quanto e 950*24?")]
})
print(r["messages"][-1].content)
```

Loop próprio ou create_agent

L Escreva o loop

- Você precisa de condição de parada específica
- Precisa instrumentar cada volta
- Precisa injetar ou podar contexto no meio
- Está ensinando alguém

C Use create_agent

- Protótipo para validar uma ideia
- Comportamento padrão serve
- Poucas ferramentas, tarefa curta
- Você vai migrar para LangGraph depois

Laboratório da manhã: migrar o NEXUS

1

Converter as duas ferramentas de ontem

calcular e ler_arquivo viram @tool. Conte as linhas antes e depois e anote a diferença.

2

Criar listar_arquivos

Nova ferramenta, com docstring escrita para o modelo: diga quando usar, não só o que faz.

3

Rodar o loop novo com as quatro paradas

Traga do dia 1 as condições de repetição e de teto de tokens; elas não vêm do framework.

4

Provocar dois erros de propósito

Argumento com tipo errado e arquivo inexistente. Anote no caderno de falhas como o modelo reagiu a cada um.

5

Escrever quatro testes de ferramenta

Sem carregar modelo nenhum. É o que vai rodar no CI a semana toda.

04

Structured output

Texto livre não encadeia.

POR QUE ISTO IMPORTA

Texto livre não encadeia.

Enquanto a saída do modelo é um parágrafo, o próximo passo do seu sistema precisa interpretar prosa: procurar a palavra aprovado, adivinhar onde está a nota, torcer para o formato não mudar. Com um objeto validado, o próximo passo lê um campo. É a diferença entre um pipeline e uma corrente de gambiarras.

with_structured_output

Você declara o formato como uma classe Pydantic, e recebe uma instância validada em vez de uma string.

O retorno é um objeto de verdade: `r.veredito` é um campo, não um trecho de texto que você localizou com regex.

O Field com `ge` e `le` não é enfeite: ele entra no schema que o modelo lê e reduz a chance de vir nota 11.

```
class Avaliacao(BaseModel):
    """Avaliacao de um relatorio."""
    nota: int = Field(ge=0, le=10)
    veredito: Literal["aprovado", "revisar"]
    problemas: list[str] = Field(
        default_factory=list)

avaliador = llm.with_structured_output(Avaliacao)
r = avaliador.invoke(
    "Avalie: 'O Brasil tem 3 estados '"
    "e fica na Europa.'")

print(r.veredito, r.nota, r.problemas)
# revisar 1 ['numero de estados errado', ...]
```

Dois mecanismos, conforme o backend

1 Tool calling forçado

- O schema vira uma ferramenta
- O modelo é obrigado a chamá-la
- Funciona em qualquer backend com tool calling
- É o caminho padrão do LangChain
- Falha quando o modelo emite JSON malformatado

2 Modo JSON ou gramática

- O servidor restringe a geração ao schema
- llama.cpp e vLLM suportam gramática
- Tokens fora do formato são impossíveis, não improváveis
- É o mais robusto quando disponível
- Exige suporte do servidor, não só do modelo

Quatro defesas, nesta ordem

1

Reduza o schema

Cinco campos rasos funcionam muito melhor que dois objetos aninhados. Modelos pequenos erram em profundidade, não em largura.

2

Dê um exemplo no prompt de sistema

Um único exemplo bem formatado costuma valer mais que três parágrafos de instrução.

3

Envolva com retry reinjetando o erro

Capture `ValidationError` e mande a mensagem do erro de volta ao modelo: sua resposta anterior falhou porque X.

4

E meça

O número de tentativas necessárias muda muito entre modelos. É um dos melhores critérios práticos para escolher modelo para agente.

Retry que reinjeta o erro de validação

```
from pydantic import ValidationError

def estruturado_com_retry(llm, modelo_saida, prompt, tentativas=3):
    estruturado = llm.with_structured_output(modelo_saida)
    msgs = [HumanMessage(prompt)]
    for i in range(tentativas):
        try:
            return estruturado.invoke(msgs)
        except (ValidationError, ValueError) as e:
            if i == tentativas - 1:
                raise
            msgs.append(AIMessage("(saida invalida)"))
            msgs.append(HumanMessage(
                f"Sua resposta anterior falhou na validacao: {e}\n"
                f"Responda de novo, corrigindo APENAS o que foi apontado."))
    raise RuntimeError("inalcancavel")
```

É a camada de feedback do dia 1 em doze linhas: o erro volta ao contexto em linguagem que o modelo consegue usar.

O que anotar no caderno de falhas

Modelo	Tentativas até validar	Schema raso	Schema aninhado
qwen3:4b			
llama3.1:8b			
outro que você tenha			

Três perguntas

1

Quando structured output é obrigatório?

Sempre que o passo seguinte do seu código fizer um if sobre a saída.

2

Por que retry funciona?

Porque o erro de validação é feedback: ele volta ao contexto em linguagem que o modelo consegue usar.

3

Primeira coisa a tentar quando falha?

Achatar o schema. Só depois exemplo no prompt, e só depois retry.

05

LCEL, callbacks e streaming

Composição, e o que dá para observar enquanto roda.

O operador pipe e a interface Runnable

Todo Runnable ganha de graça: invoke, batch, stream, ainvoke e configuração em tempo de execução.

Use LCEL para pipelines determinísticos: prompt, depois modelo, depois parser.

Para fluxo com ramificação, loop e estado, a ferramenta certa é o LangGraph, e é para lá que vamos amanhã.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_messages([
    ("system", "Voce e {papel}. Tom {tom}."),
    ("placeholder", "{historico}"),
    ("human", "{pergunta}"),
])

cadeia = prompt | llm | StrOutputParser()

cadeia.invoke({"papel": "analista", "tom": "direto",
              "historico": [], "pergunta": "O que e mediana?"})
```

Cadeias sequenciais com saída em JSON

```
class Resumo(BaseModel):
    titulo: str
    pontos_chave: list[str] = Field(description="3 a 5 itens")
    sentimento: str

parser = JsonOutputParser(pydantic_object=Resumo)

prompt_resumo = ChatPromptTemplate.from_messages([
    ("system", "Resuma o texto no formato pedido.\n{formato}"),
    ("human", "{texto}"),
]).partial(formato=parser.get_format_instructions())

prompt_tradacao = ChatPromptTemplate.from_template(
    "Traduza pontos_chave de {resumo} para ingles.")

# passo 1 (dict = RunnableParallel) -> passo 2 -> passo 3
cadeia = ({ "resumo": prompt_resumo | llm | parser }
          | prompt_tradacao | llm | StrOutputParser())
```

Um dicionário dentro de uma cadeia LCEL vira um RunnableParallel implícito: cada chave roda sua sub-cadeia, e o resultado alimenta o passo seguinte com aquele nome.

Quando LCEL, quando grafo

L LCEL resolve bem

- Sequência fixa de passos
- Sem ramificação condicional
- Sem loop de correção
- Sem estado que sobrevive à chamada
- Exemplo: resumir, depois traduzir

G Precisa de LangGraph

- Caminhos diferentes conforme o resultado
- Loop até um critério de qualidade
- Pausa para aprovação humana
- Estado persistido entre execuções
- Exemplo: o NEXUS inteiro, a partir de amanhã

Ver a resposta enquanto ela é gerada

Todo Runnable expõe stream. Em modelo local, isso muda completamente a percepção de velocidade: o primeiro token sai em menos de um segundo, mesmo que a resposta inteira leve trinta.

Streaming é apresentação, não arquitetura. A decisão de chamar ferramenta só existe quando a mensagem termina, então o loop do agente não muda.

No dia 3 o LangGraph estende isso: dá para transmitir também os passos do grafo, e não só os tokens.

```
for pedaco in cadeia.stream({"pergunta": "..."}):  
    print(pedaco, end="", flush=True)  
  
# no nível do modelo:  
for tok in llm.stream("Explique embeddings"):  
    print(tok.content, end="", flush=True)  
  
# versao assincrona, para servidor web:  
async for pedaco in cadeia.astream(entrada):  
    await enviar(pedaco)
```


Instrumentar sem sujar o código de negócio

Um callback recebe eventos do ciclo de vida: início e fim de modelo, início e fim de ferramenta, erro. É o gancho de observabilidade do LangChain.

Isto é a instrumentação do dia 1 sem poluir a função de negócio: em vez de prints espalhados, um objeto que você liga e desliga por configuração.

No dia 4 trocamos este handler artesanal por LangSmith ou Langfuse, que fazem o mesmo com interface e persistência.

```
from langchain_core.callbacks import BaseCallbackHandler

class Contador(BaseCallbackHandler):
    def __init__(self):
        self.chamadas, self.tokens = 0, 0

    def on_llm_end(self, resposta, **kw):
        self.chamadas += 1
        uso = resposta.llm_output or {}
        self.tokens += uso.get("token_usage", {}).get(
            "total_tokens", 0)

    def on_tool_start(self, ferramenta, entrada, **kw):
        print(f" -> {ferramenta['name']}({entrada[:60]})")

c = Contador()
cadeia.invoke(entrada, config={"callbacks": [c]})
print(c.chamadas, c.tokens)
```

CHECKPOINT DO BLOCO

Callback observa. Hook decide.

Guarde essa frase: ela separa a camada de feedback da camada de constraints, e evita o erro clássico de tentar impedir comportamento com um mecanismo que só sabe registrar. Amanhã, no LangGraph, a diferença vira código.

06

Fundamentos por baixo do RAG

Por que RAG funciona, antes de indexar a primeira pasta.

O PROBLEMA QUE RAG RESOLVE

**Não peça ao modelo que lembre o fato.
Busque o fato e coloque na frente dele.**

É a mesma lição do dia 1, agora aplicada a documentos: o modelo não tem acesso ao seu mundo. RAG converte uma tarefa em que LLM é ruim, lembrar um fato específico, numa tarefa em que ele é muito bom, ler e resumir o que está na tela. E por isso toda resposta do NEXUS exige a fonte: a citação é a prova de que veio de recuperação, e não de memória.

Fine-tuning e RAG resolvem problemas diferentes

	Fine-tuning	RAG
O que muda	Os pesos do modelo	O que entra no contexto
Conhecimento novo	Aprendido nos exemplos de treino	Buscado em tempo de consulta
Atualizar o conhecimento	Retreinar: caro e lento	Trocar o documento: imediato
Rastreabilidade	Nenhuma; é intuição do modelo	Alta; dá para citar a fonte
Bom para	Ensinar um estilo ou formato	Ensinar fatos que mudam

Embeddings e métricas de similaridade

Um embedding transforma texto num vetor tal que textos com significado parecido geram vetores próximos. Próximo precisa de uma métrica.

Cosseno mede o ângulo entre os vetores e ignora a magnitude. É o padrão de fato para embeddings de texto, porque a magnitude costuma refletir o tamanho do texto, não o significado.

Euclidiana mede a distância em linha reta. Mais comum em embeddings de imagem, ou quando os vetores já estão normalizados, caso em que as duas ordenam igual.

O Chroma usa cosseno por padrão.

```
import numpy as np

def similaridade_cosseno(a, b):
    # angulo entre vetores. [-1, 1], 1 = identico
    return np.dot(a, b) / (
        np.linalg.norm(a) * np.linalg.norm(b))

def distancia_euclidiana(a, b):
    # distancia em linha reta. 0 = identico
    return np.linalg.norm(
        np.array(a) - np.array(b))
```

Três estratégias de chunking

Estratégia	Como corta	Ganha	Perde
Fixo	N caracteres exatos	Simples e rápido	Corta frases e tabelas ao meio
Recursoivo	Tenta parágrafo, depois linha, depois espaço	Respeita a estrutura	Ainda corta mal texto sem estrutura
Com sobreposição	Qualquer um acima, repetindo N caracteres entre trechos	A frase cortada aparece inteira em algum trecho	Infla o índice

Metadados e PDF

Metadado não é enfeite: source vira a citação da fonte na resposta, e filtrar por metadado antes da busca é mais barato e mais preciso que buscar em tudo e descartar depois.

A pasta docs/ do NEXUS é só Markdown, mas a maioria dos documentos do mundo real é PDF. O padrão muda pouco: o resto do pipeline, splitter, embedding e Chroma, é idêntico.

PDF escaneado, que é imagem sem texto selecionável, precisa de OCR antes.

```
from langchain_community.document_loaders \
    import PyPDFLoader

loader = PyPDFLoader("contrato.pdf")
paginas = loader.load() # 1 Document por pagina
print(paginas[0].metadata)
# {'source': 'contrato.pdf', 'page': 0}

# filtrar por metadado na busca:
banco.similarity_search(consulta, k=4,
    filter={"fonte": "2024_relatorio.md"})
```


FECHO DO BLOCO

Chunk, métrica e filtro são decisões de projeto. A biblioteca só executa a decisão.

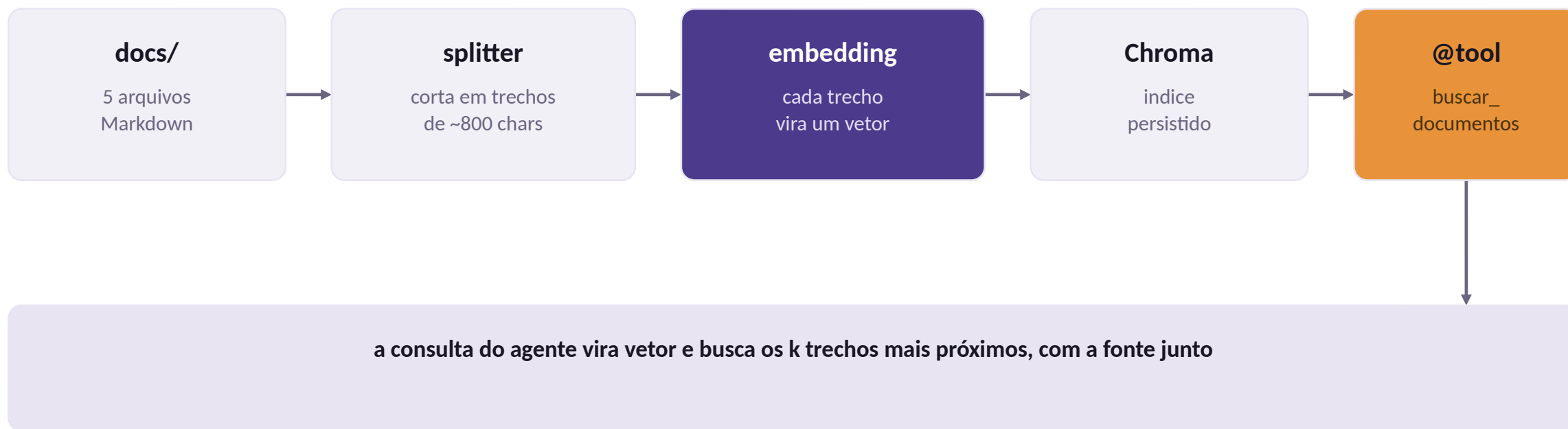
É o mesmo padrão do dia inteiro: o framework tira o trabalho mecânico e devolve para você exatamente as escolhas que importam. Quem aceita o padrão sem medir está terceirizando a decisão para quem escreveu o exemplo do README.

07

RAG local

Dando documentos ao agente sem estourar o contexto.

Do arquivo ao trecho relevante



Os quatro primeiros passos acontecem uma vez, na indexação. Só o último acontece a cada pergunta.

Dois parâmetros que merecem discussão

`chunk_size`: pedaço grande carrega contexto mas dilui o significado no vetor; pedaço pequeno é preciso mas perde o entorno. De 500 a 1000 caracteres é um bom começo para português.

`separators`: cortar em cabeçalho de Markdown respeita a estrutura do documento. Cortar no meio de uma frase produz trechos que não fazem sentido isolados, e o modelo recebe lixo bem formatado.

```
docs = DirectoryLoader("docs/", glob="**/*.md",
                       loader_cls=TextLoader).load()

partes = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=120,
    separators=["\n## ", "\n### ", "\n\n", "\n", " "],
).split_documents(docs)

banco = Chroma.from_documents(
    partes, emb, persist_directory=".chroma")
```

Metadado de origem é requisito, não enfeite

```
@tool(args_schema=BuscaArgs)
def buscar_documentos(consulta: str, k: int = 4) -> str:
    """Busca trechos relevantes na base local de documentos.

    Use ANTES de afirmar qualquer coisa sobre o conteúdo dos documentos.
    """
    achados = banco.similarity_search(consulta, k=max(1, min(k, 8)))
    if not achados:
        return "Nenhum trecho relevante encontrado. Tente outros termos."
    return "\n\n--\n\n".join(
        f"[fonte: {Path(d.metadata['source']).name}]\n{d.page_content}"
        for d in achados)
```

Sem a fonte, o agente não consegue citar de onde tirou a informação, e um relatório sem fonte é inútil para quem vai decidir com base nele.

Diagnóstico rápido de RAG

Sintoma	Causa provável	O que tentar
Traz trecho de assunto vizinho	chunk_size grande demais	Reduzir e ajustar separadores
Traz o mesmo trecho repetido	chunk_overlap alto demais	Baixar para 10 a 15 por cento
Não acha o que está escrito literalmente	Busca vetorial não é busca textual	Combinar com busca por palavra-chave
Resultados ótimos em inglês, ruins em português	Modelo de embedding monolíngue	Trocar por um multilíngue
Resultados de versão antiga do documento	Índice não reconstruído	Apagar o persist_directory e reindexar

Laboratório da tarde: RAG no NEXUS

1

Indexar a pasta docs/

Cinco documentos da Cooperativa Vale Verde, com `chunk_size` 800 e sobreposição 120, persistido em `.chroma`.

2

Criar `buscar_documentos` com Pydantic

Com limite de `k`, filtro por fonte e a citação da origem em cada trecho retornado.

3

Comparar três `chunk_size`

400, 800 e 1500 na mesma pergunta. Anote qual trouxe o trecho certo e por quê.

4

Responder uma pergunta que exige buscar E calcular

Some o faturamento de 2023 e 2024 e diga a média. O agente precisa encadear as duas ferramentas.

5

Structured output no fim

Devolva um Relatório validado, com resposta, lista de fontes e nível de confiança.

08

Fechamento

O que entregar hoje, e a pergunta de amanhã.

O que ficou pronto

M Manhã: o NEXUS migrado

Ferramentas com @tool, docstring escrita para o modelo, Pydantic validando antes da execução, quatro testes rodando sem carregar modelo, e o loop com as quatro condições de parada.

T Tarde: RAG e structured output

A pasta docs/ indexada, buscar_documentos citando a fonte, a comparação de três chunk_size, uma resposta que exigiu buscar e calcular, e um Relatório validado no fim.

Erros clássicos do dia 2

Sintoma	Causa	Correção
args_schema vazio	Falta anotação de tipo	Anote todos os parâmetros
Modelo ignora a ferramenta	Docstring diz o que faz, não quando usar	Reescreva com o gatilho de uso
with_structured_output falha sempre	Schema aninhado demais	Achate o schema e dê um exemplo
Busca traz trechos irrelevantes	chunk_size grande demais	Reduza e ajuste os separadores
Chroma devolve resultados antigos	persist_directory reaproveitado	Apague .chroma ao reindexar
ToolMessage sem par	tool_call_id faltando	Um ToolMessage por tool call, sempre

O caderno de falhas

1

Uma linha por falha real

O que você pediu, o que o agente fez, e qual componente do harness estava errado.

2

A correção vira regra ou teste

Se dá para virar teste, vira teste. Se não dá, vira linha de instrução. É a escola do Hashimoto, do dia 1.

3

Revise no fim de cada dia

Cinco minutos. É o que transforma tentativa e erro em conhecimento acumulado.

4

Leve para o trabalho

É a prática mais barata deste curso e a que mais rende depois que a semana acabar.

As cinco camadas, e o que fizemos hoje

Camada	O que construímos hoje	Onde aprofunda
Instructions	Docstrings escritas para o modelo, prompt templates	Dia 3, no prompt de cada nó
Constraints	Pydantic validando antes de executar, ToolRuntime	Dia 3, com hooks que negam
Feedback	Retry que reinjeta o erro, testes de ferramenta	Dia 4, com agente avaliador
Memory	Ainda nada; o histórico morre com a função	Dia 3, com checkpointer
Orchestration	LCEL para sequência fixa, e só	Dia 3 e 4, com grafo

PARA AMANHÃ

O nosso loop ainda é um for com max_passos.

Como fazer para pausar no meio e pedir aprovação humana? Para retomar amanhã de onde parou? Para ter dois caminhos diferentes conforme o que a ferramenta devolveu? Para rodar três análises em paralelo? Com for e if, tudo isso vira uma função de trezentas linhas que ninguém consegue depurar. Amanhã trocamos o for por um grafo.

Dois textos e uma prática

1

Documentação do LangChain sobre tools

Leia a parte de ToolRuntime e de tratamento de erro; é o que menos aparece em tutorial.

2

Anthropic, Writing effective tools for agents

O melhor texto curto sobre description e Agent-Computer Interface.

3

Prática: reescreva uma docstring do trabalho

Pegue uma função real do seu projeto e escreva a description como se fosse para o modelo. Traga amanhã.